

**Arbol Binario de Búsqueda (ABB):** Desarrollar la clase que implementa un árbol binario de búsqueda que permite almacenar números enteros y realizar operaciones sobre ellos.

La clase **ABB** tiene los siguientes atributos:

- **int n:** atributo que almacena de forma privada el número de nodos del árbol.
- **Nodo \*raiz.** Puntero a una variable de tipo struct **Nodo** que se considerará como la raíz del árbol. Si el puntero apunta a NULL indicará que el árbol está vacío. Esta estructura tiene los siguientes campos:
  - o **contenido.** Elemento de tipo entero donde se guardará los números
  - o **padre.** Puntero apuntando al padre o a NULL si es la raíz.
  - o **hijoIzquierdo.** Apunta al hijo izquierdo, o NULL si no tiene hijo.
  - o **hijoDerecho.** Apunta al hijo derecho, o NULL si no tiene hijo.

Para esta clase deberán implementarse los siguientes métodos públicos:

- **Constructor.** Crea un árbol vacío.
- **Destructor.** Se encargará de liberar la memoria que fue reservada de forma dinámica para almacenar todos los nodos del árbol, llamará de forma interna al método recursivo **eliminarSubArbol**.
- **void leerArbol().** Este método lee un árbol a partir de la entrada llamando internamente al método **leerSubarbol**. (Este método no es necesario implementarlo)
- **void imprimir().** Este método realiza una representación del recorrido en in orden del árbol actual usando el método recursivo **imprimirRecorrido**. (Este método no es necesario implementarlo)
- **void insertar(int nuevoElemento).** Inserta un nuevo elemento en el árbol como una hoja en la posición que le corresponde para ello utiliza internamente el método **buscarHueco**.
- **Nodo \*buscar(int elementoABuscar).** Método público que busca un elemento en el árbol binario de búsqueda que coincida con el valor indicado en **elementoABuscar**. Internamente utilizada el método privado **buscarRecursivo**.
- **void eliminar(int elementoAEliminar).** Elimina el elemento del árbol que coincide con el valor indicado por **elementoAEliminar**. Si el elemento se encuentra en una hoja se deberá eliminar únicamente dicho elemento. En el caso de que el nodo sea interno deberá, intercambiarse por otro nodo de la siguiente manera:
  - o Si el nodo sólo tiene un hijo izquierdo, se deberá buscar el máximo elemento del subárbol cuyo padre es el hijo izquierdo, para ello se utilizará el método interno **buscarMaximo**.
  - o Si el nodo sólo tiene un hijo izquierdo, se deberá buscar el mínimo elemento del subárbol cuyo padre es el hijo derecho, para ello se utilizará el método interno **buscarMinimo**.

- o Si el nodo tiene ambos hijos, elegiremos el hijo que tenga una mayor altura mediante el método **alturaNodo** y a continuación realizaremos las operaciones indicadas anteriormente.

Debido a que puede ocurrir que el máximo o el mínimo no sean nodos hojas será necesario realizar un método recursivo **eliminarNodo** que implemente este proceso para ir intercambiando nodos hasta llegar a un nodo hoja.

- **bool esABB()**. Indica si el árbol cumple las condiciones para ser un árbol binario de búsqueda. Este método llamará internamente al método recursivo **esSubABB**.
- **bool esAVL()**. Indica si el árbol cumple las condiciones para ser un árbol AVL. Este método llamará internamente al método recursivo **esSubAVL**.

Para esta clase deberán implementarse los siguientes métodos privados que implementan diferentes funciones recursivas:

- **void eliminarSubarbol(Nodo \*raizSubarbol)**. Método que elimina de forma recursiva los nodos que cuelgan de **raizSubarbol**.
- **Nodo \*leerSubarbol(Nodo \*padre)**. Método que permite crear un árbol de forma recursiva a partir de la entrada. (Este método no es necesario implementarlo).
- **void imprimirRecorrido(Nodo \*raizSubarbol)**. Método recursivo que representa el árbol mediante su recorrido en in orden (Este método no es necesario implementarlo).
- **Nodo \*buscarHueco(Nodo \*raizSubarbol, int elementoAInsertar)**. Método que busca recursivamente el padre del hueco donde podemos insertar un nuevo elemento dado a partir del subárbol que tiene como raíz **raizSubarbol**.
- **Nodo \*buscarRecursivo (Nodo \*raizSubarbol, int elementoABuscar)**. Método que busca recursivamente el elemento que coincide con **elementoABuscar** a partir del subárbol que tiene como raíz **raizSubarbol**.
- **Nodo \*buscarMaximo(Nodo \*raizSubarbol)**. Método que busca el máximo de un subárbol (ir por la rama derecha hasta llegar a la última hoja). Si no hay ningún hijo derecho más devolverá la raíz.
- **Nodo \*buscarMinimo(Nodo \*raizSubarbol)**. Método que busca el mínimo de un subárbol (ir por la rama izquierda hasta llegar a la última hoja). Si no hay ningún hijo izquierdo más devolverá la raíz.
- **void eliminarNodo(Nodo \*nodoParaEliminar)**. Este método implementa el proceso indicado anteriormente en eliminar pero de forma recursiva hasta que el nodo eliminado sea una hoja.
- **Int alturaNodo(Nodo \*raizSubarbol)**. Método que permite calcular la altura de un nodo en un árbol. Si el árbol es vacío se considera que la altura es -1 y si es un nodo hoja 0.
- **bool esSubABB(Nodo \*raizSubarbol)**. Comprueba de forma recursiva si el subárbol cumple las condiciones para ser un árbol binario de búsqueda.

- **bool esSubABB(Nodo \*raizSubarbol).** Comprueba de forma recursiva si el subárbol cumple las condiciones para ser un árbol de tipo AVL.

Utilizar todos los ficheros proporcionado a través de Blackboard para realizar las pruebas necesarias y para enviar al corrector automático. Este programa permite realizar las siguientes operaciones:

- N: permite crear un nuevo árbol.
- A: permite insertar un valor en el árbol.
- E: permite eliminar un valor del árbol.
- I: imprime el recorrido en in orden del árbol.
- L: lee un árbol por la entrada.
- B: comprueba si es un árbol binario de búsqueda
- V: comprueba si es un árbol AVL
- F: liberar la memoria ocupada por el árbol.
- S: salir.

NOTA: Para cada uno de los métodos implementados se deberá incluir una pequeña descripción de su funcionamiento, sus precondiciones mediante `assertdomjudge` si las hubiera y el análisis de su complejidad temporal y espacial. Esta información deberá incluirse en la cabecera de cada función.